Project Proposal

Distributed Middleware for AI Inference

Team Members

Yingyu Gu (Yukki) and Joseph Fujimoto

I. Problem Statement

Traditional middleware systems act as an intermediary between clients and services, coordinating communication between services and remote resources. Typically, a system receives a client request and routes the request to a valid and healthy service through mechanisms such as naming and service discovery, communication, group membership, etc., while maintaining reliability through concurrency control, security, and fault tolerance mechanisms.

With the introduction of AI services, this pipeline becomes more complex because workers may host different types of models, have different hardware capabilities, and operate under rapidly changing levels of resource availability. The new middleware system must, therefore, be able to evaluate worker capabilities, health, and current resources when discovering and routing requests.

Our project explores the design and implementation of a distributed middleware layer for AI inference that supports service registration and discovery, dynamic membership, failure detection, replicated state, concurrency-safe updates, and load-aware request routing across multiple inference workers. The goal is to implement a scalable and fault-tolerant system with dynamic, resource-aware mechanisms, while allowing clients to access AI services without directly managing location, state, and resource availability.

II. Motivation

As the use of distributed AI services increases, an efficient coordination middleware system becomes crucial because poor resource management or ineffective routing can directly impact latency, reduce throughput, and waste computing resources. AI workloads are also sensitive to inefficiencies because their performance can vary significantly across diverse hardware such as CPUs or GPUs.

Building a middleware layer that can effectively respond to these conditions is challenging because routing and allocation decisions must be made using these changing conditions and information from requests. Due to the fact that the information is distributed, the middleware system must account for differences in request requirements, worker load, resource availability, service capabilities, and failures, while still making decisions quickly. In addition, the system must ensure that decision-making and resource awareness do not introduce excessive coordination overhead by balancing responsiveness, consistency, and fault tolerance. The coordination layer should not become a bottleneck to the dynamic system.

A solution to this problem would be beneficial for developers, system operators, and organizations deploying distributed AI applications. Developers would be able to use a simpler interface to access AI inference services without the need to directly manage the backend infrastructure, while system operators are able to have better control over workload distribution, resource utilization, scalability, and failover recovery. By extending the traditional service discovery with stronger resource awareness and flexibility in resource allocation, the system can make distributed AI infrastructure easier to manage and more efficient, which is especially important in this growing age of AI services.

III. Related Literature and Previous Work

The architecture of a distributed middleware system for AI inference aligns three established domains of research in distributed systems: distributed AI inference sharing, decentralized cluster membership/discovery, and fault-tolerant replication frameworks.

Recent breakthroughs in Large Language Model (LLM) architectures and massive deep learning workloads have shifted the focus of model serving from single-node optimizations to distributed cluster-wide orchestration. While systems like Preble [1] use hierarchical scheduling to optimize KV cache reuse and computational load balancing across dynamic GPU networks, they typically assume a stable, pre-existing orchestration fabric. This leaves an open requirement for a decentralized middleware layer that can actively track worker health and route requests without relying on centralized, single-point-of-failure (SPOF) dispatchers.

Distributed AI inference serving requires scalable, fault-tolerant middleware to handle dynamic service discovery and load-aware request routing [1]. While traditional platforms rely on centralized registries that create performance bottlenecks during high-frequency worker updates, recent advancements look toward decentralized architectures. Specifically, Mueggenburg et al. [2] developed a decentralized microservice orchestrator for smart manufacturing systems, proving how localized coordination can eliminate SPOFs. This project builds on those decentralized coordination concepts, adapting them to AI inference workloads by using gossip protocols and load-aware allocation to route client requests efficiently without a central bottleneck.

In a centralized registry system, services register with a single directory, which can become a SPOF if it disrupts service discovery and communication across the system. To address this limitation, Chen et al. [3] propose a decentralized blockchain-based distributed service registry and discovery mechanism for a 6G core network. Their architecture replicates service information across distributed registry nodes and supports registration, discovery, consensus, and heartbeat monitoring for reliability. This work relates to our project because our AI inference workers also need to register their capabilities and be discovered by clients or middleware services. However, our system will use a lightweight replicated registry and additionally consider worker load and resource availability when selecting inference services.

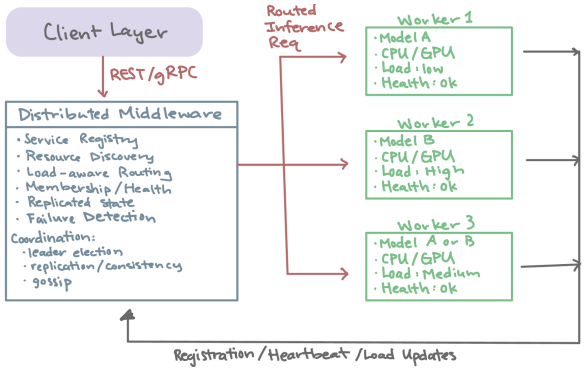
In addition to extending service discovery, a reliable middleware system must maintain a correct shared state under concurrent updates and failures. Yamada and Nemoto [4] propose ScalarDL, a database-agnostic middleware designed to maintain correctness across replicated transactional databases using transaction ordering, versioning, concurrency control, and validation while detecting Byzantine faults. Their system achieved near-linear (91%) scalability even when replica nodes increased [4]. Unlike [3]'s blockchain-based approach, ScalarDL focuses on coordination between a small number of replicas without relying on large-scale replicated consensus, which relates to our project needs. However, instead of implementing the entire Byzantine fault-detection protocol, we can use a lighter management system with versioned state, concurrency control, and recovery principles to maintain a reliable worker and service state in our distributed AI middleware.

IV. Proposed Approach and Methodology

Our system will use a 3-part architecture consisting of clients, distributed middleware nodes, and AI inference workers.

- **Client Layer:** Clients send requests for a specific AI service or model through middleware
- **Middleware Layer:** Middleware nodes maintain a distributed registry of workers, track health and resource availability, perform service discovery, and route requests to suitable workers.
    - State will be replicated across middleware nodes to avoid a SPOF.
- **Inference Workers:** Workers host AI models or simulated inference workloads, register their capabilities, report health/load information, and may dynamically join or leave the system.
- **Distributed Algorithms:** The system will use resource discovery, resource allocation, replication/consistency, and concurrency control. Gossip and leader election may also be used for membership and failure recovery.
- **Fault Handling:** Failed or overloaded workers will be detected, and requests rerouted. Middleware failures will be handled through replicated state and coordination mechanisms.
- **Evaluation:** We will measure latency, throughput, recovery time, load distribution, and scalability as the number of clients and workers increases.

Implementation and Deployment: Components will communicate through REST or gRPC and run as separate processes or Docker containers across local machines or cloud VMs.



V. Project Timeline and Milestones

Target Date	Milestone	Tasks
Week 1	Proposal & Design	Finalize architecture, algorithms, interfaces, and environment.
Week 2	Core Prototype	Implement registration, discovery, routing, and worker communication.
Week 3	Coordination	Add heartbeats, failure detection, replicated state, and versioned updates.
Week 4	Resource-Aware Routing	Add model/resource metadata and routing strategies.
Week 5	Fault Tolerance	Implement retries, rerouting, join/leave handling, and failure recovery.
Week 6	Evaluation	Measure latency, throughput, scalability, recovery time, and load distribution.
Week 7	Integration	Fix bugs, optimize coordination, and finalize deployment.
Week 8	Finalization	Complete testing, results, presentation, and report.
12/3/2026	Presentation and Code	Submit code and present demo.
12/4/2026	Final report	Submit the final report.

VI. Resources Required

- Containerization: Docker
- Version Control: GitHub
- Programming Language: Python or Go
- Communication: gRPC or REST
- Persistent Storage: SQLite or PostgreSQL
- Deployment: Local machines or cloud VMs
- AI Workloads: Lightweight inference models or simulated workloads
- Hardware: GPU access if available, but not required

References

[1] V. Srivatsa, Z. He, R. Abhyankar, D. Li, and Y. Zhang, "Preble: Efficient Distributed Prompt Scheduling for LLM Serving," International Conference on Learning Representations, vol. 2025, pp. 37057–37082, May 2025, Accessed: Oct. 08, 2026. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/hash/5bc342f48de8264779952fac378f96dc-Abstract-Conference.html

[2] B. Mueggenburg, Z. Perkin, and X. Xu, "Development of a Microservice Orchestrator for decentralised smart manufacturing systems," Journal of Manufacturing Systems, vol. 86, pp. 579–585, June 2026, doi: 10.1016/j.jmsy.2026.04.006.

[3] S. Chen, Y. Lin, R. Zhang, and Y. Liu, "Blockchain-Based Distributed Service Registry and Discovery in 6G Core Network," in 2023 8th International Conference on Computer and Communication Systems (ICCCS), IEEE, Apr. 2023, pp. 714–718. doi: 10.1109/icccs57501.2023.10151096.

[4] H. Yamada and J. Nemoto, "Scalar DL," Proceedings of the VLDB Endowment, vol. 15, no. 7, pp. 1324–1336, Mar. 2022, doi: 10.14778/3523210.3523212.

1) **Preble: Efficient Distributed Prompt Scheduling for LLM Serving**
   Available via the ICLR 2025 Proceedings.
2) **Development of a Microservice Orchestrator for decentralised smart manufacturing systems**
   Available via ScienceDirect.
3) **Blockchain-Based Distributed Service Registry and Discovery in 6G Core Network**
   Available via IEEE Xplore.
4) **Scalar DL**
   Available via the ACM Digital Library or open access through the VLDB Endowment.